# Introduction to Programming

# Contents

# What is programming?

# What is programming? I – Definition



- “Programming is the formulation of a problem in the form of order instructions with the help of a language, which is translated into commands, that can be executed by a computer.“

```python
def Dihedral(xyz1,xyz2,xyz3,xyz4):
    return (360/(2*math.pi))*BP.calc_dihedral(BP.Vector(xyz1.iloc[0,:]),BP.Vector(xyz2.ilo

Models = ['ModBase','SwissModel','T0863','T0872','T0886','T0892','T0917','T0944']
Strucs = ['Single_Test','Single_Training','XRay5000']
Dirs = ['ModBase','SwissModel','XRay5000']
AA = ['ALA','ARG','ASN','ASP','CYS','GLN','GLU','HIS','ILE','LEU','LYS','MET','PHE','PRO',
Output = pd.DataFrame(data=0,index=AA+['All','Strucs'],columns=Dirs)

for Dir in Dirs:
    os.chdir(Dir)
    if Dir in Models: Files = glob.glob('*.pdb')
    else: Files = glob.glob('*_clean.pdb')
    for F in Files:
        Count = 0
        ppdb = PandasPDB()
        Struc = ppdb.read_pdb(F)
        Chains = list(set(Struc.df['ATOM']['chain_id']))
        Chains.sort()
        Numbers = list(set(Struc.df['ATOM'][Struc.df['ATOM']['chain_id'] == Chains[0]]['re
        Numbers.sort()
        for Nr in Numbers:
            Amino = list(set(Struc.df['ATOM'][(Struc.df['ATOM']['residue_number'] == Nr) &
            if Amino in AA:
                C = Struc.df['ATOM'][(Struc.df['ATOM']['atom_name'] == 'C') & (Struc.df['A
                CA = Struc.df['ATOM'][(Struc.df['ATOM']['atom_name'] == 'CA') & (Struc.df[
                N = Struc.df['ATOM'][(Struc.df['ATOM']['atom_name'] == 'N') & (Struc.df['A
                CB = Struc.df['ATOM'][(Struc.df['ATOM']['atom_name'] == 'CB') & (Struc.df[
                if C.size > 1 and CA.size > 1 and N.size > 1 and CB.size > 1:
                    Zeta = Dihedral(CA,N,C,CB)
                    if pd.notnull(Zeta):
                        if Zeta < 0:
                            Output.loc[Amino,Dir] += 1
                            Count += 1
                            Output.loc['All',Dir] += 1
        if Count > 0: Output.loc['Strucs',Dir] += 1
```

# What is programming? II – Explanation



- Programming means to write scripts that can be executed by a computer to fulfill certain tasks
- These tasks can include statistical calculations, analyzing sequences, reading and deconstructing files or communicating with databases
- It allows us to automate these tasks and thus makes repetitive work much easier

# What is programming? III – Job of a programmer [1]



- A programmer, someone who creates programs for automating certain tasks, has several jobs at once:

1. Development of source code for programs
2. Testing programs to make sure everything works as intended
3. Debugging programs, that means searching and correcting errors in the source code
4. Source code maintenance, to make sure that the program works at all times
5. Optimization of Calculation Time and Memory Management

# A short History

# History I – Beginnings [1]



- In the 9th century, a programmable music sequencer was invented by the Persian Banu Musa brothers
- In the 9th century, the Arab mathematician Al-Kindi described a cryptographic algorithm for deciphering encrypted code
- In 1206, the Arab engineer Al-Jazari invented a programmable drum machine
- In 1801, the Jacquard loom could produce entirely different weaves by changing the "program" – a series of pasteboard cards with holes punched in them

# History II – The first program [1]

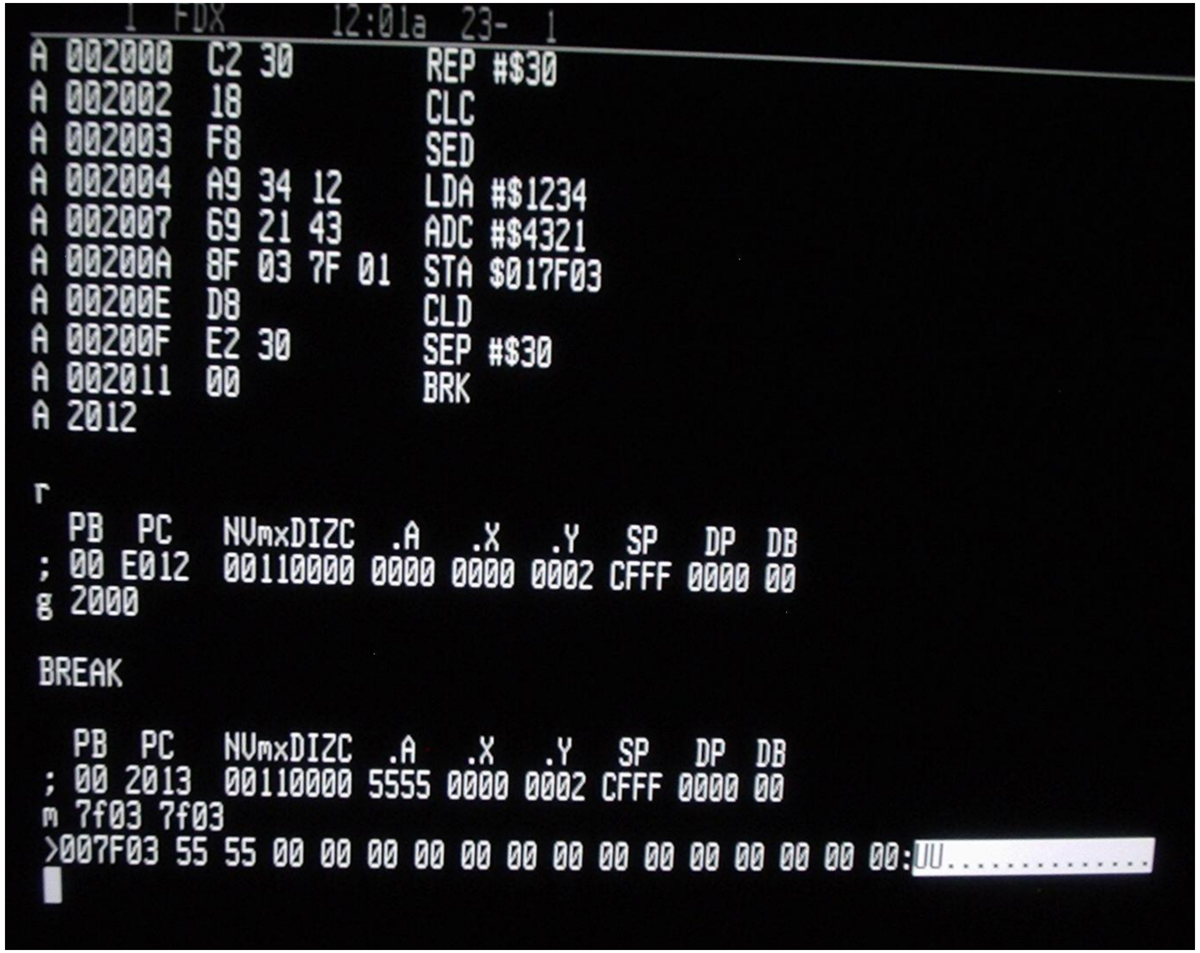


- The first “real” computer program is generally dated to 1843, when mathematician Ada Lovelace [2] published an algorithm to calculate a sequence of Bernoulli numbers

# History III – Machine Language [1]



- Early programs were written in Machine code
- Were included in the instruction set of a particular machine
- In most cases in a form of binary notation (only 1 and 0)
- Were later replaced by assembly languages that allowed to specify instructions in text form



# History IV – Compiler languages [1]



- The first compiler language, the A-0 Sytem [3] was developed in 1952 by Grace Hopper [4]
- The first widely used high-level language FORTRAN [5] came out in 1957
- Compiled languages allow the programmer to write programs in terms that are syntactically richer, and more capable of abstracting the code

# From code to program

# From code to program – Steps



| Step | Description |
| --- | --- |
| Idea | The task you want to be automated<br><br>The analysis you want to execute |
| Source Code | The script you write |
| Compiler / Interpreter | Converts your script into a program |
| Machine Language | The resulting program |
| Execution | The computer fullfills the tasks given |

# Why are they called programming languages?

# Programming Language I – What is a Language?



- Essential form of communication
- Consists of specific letters (alphabet)
- Grammar defines membership of a language
- Sentence structure is determined via syntax
- Semantics is language independent – describes the contents of the spoken

# Programming Language II – Why no intuitive language?



- A computer “thinks” different than a human
- Computers are dumb – they can only do exactly as they’re told
- Languages always try to get even more intuitive ➔ languages are more difficult to read

# Programming Language III – Specifics



- Hand over commands to the compiler / computer
- Programing languages differ in their syntax and their powerfulness
- Are explicit
- Aren‘t spoken

# Programming Language IV – What is Syntax?



- Dictates if a command is recognized as a part of the language
- Syntax errors prevent the translation of the source code into machine code
- Are detected by the compiler / interpreter
- Use IDEs (Integrated Development Environments) like Thonny, Spyder, ATOM or Visual Studio to prevent easy syntax errors, like missing brackets or punctuation

# Programming Language V – What is Semantics?



- Refers to the contents of the commands
- Semantics errors can only be found by the programmer and not by the compiler / interpreter
- Often hard to spot
  - Plan everything as good as possible **prior to** starting with the programing

# Programming Language VI – Punctuation on programming



- Normal Languages
- Normal sentences end with a period / exclamation mark / question mark
- Sentences are ordered with the help of paragraphs

- Programming Languages
- For computers instructions end with special characters like a semi-colon or a curly bracket
- Are structured by blocks that have the same indentation level

# Parts of Programs

# Parts of programs



- A program consists of many different parts that each fulfill a specific role
- Over the course of the next weeks we will discuss the all of these in detail
- In general a program consists of the following:
  - Variables
  - Functions
  - Classes
  - Algorithms
    - Normal commands
    - Branchings
    - Loops

# Computer and Instructions

# Baking with a PC I

- Bake the cake in the oven at 180°C for 20 minutes

# Insert: Computers are dumb I



- For a human these instructions are clear
- … but not for a PC
- Computers need much more precise instructions
- If not:

# Insert: Computers are dumb II



# Insert: Computers are dumb III



# Baking with a PC II



- Bake the cake in the oven at 180°C for 20 minutes
  - ➡ There is no clarification that the door of the oven has to be opened beforehand
  - ➡ In the worst case scenario the door of the oven is damaged or the cake (“corrupted”)
  - ➡ For simple programs this is rather rare

# Baking with a PC III



- Open the door of the oven
- Put the cake into the oven
- Close the door of the oven
- Bake the cake in the oven at 180°C for 20 minutes
    - ➡ The computer could stop the program if the door is already open

# Baking with a PC IV



- If the oven door is closed, open the oven door and put the cake into the oven
- When not put the cake directly into the oven
- Close the door of the oven
- Bake the cake in the oven at 180°C for 20 minutes
  - ➡ This runs even if something is still in the oven.
  - ➡ If there is an old cake in the oven it could catch fire

# Baking with a PC V



- If the oven door is closed, open the oven door
  - When the oven is empty put the cake into the oven
  - If not empty the oven and put the cake into the oven
- When not put the cake directly into the oven if the oven is empty
  - If not empty the oven and put the cake into the oven afterwards
- Close the door of the oven
- Bake the cake in the oven at 180°C for 20 minutes

# Think beforehand



- Prior to saving a file – is there enough memory?
- Prior to opening a file – does this file exist?
- Prior to working with a variable –
  - is the variable declared?
  - has the variable the right type?
- etc…

# Exercise 01: Roasting a Steak



- Write a Guide for a Computer to roast a steak
- Follow the Example we had for baking a Cake
- You need to make sure that you have a Pan, that you have oil in your Pan, that the steak is in the pan, that the fire of the Stove is on and you have to flip over your Steak after a specific time
- Last but not least your steak has to rest for 2 Minutes before serving
- Don’t include any other questions or problems in this process aside of the ones mentioned in the text

# History of Python

# History of Python [6]



- Developed by Guido van Rossum in the early 90’s
- As successor of the language ABC
- Name inspired by Monty Python
- First full version in 1994
- Since 2008 two different versions 2.7.18 and 3.10.0
- Python 2 is no longer supported

# Why should you learn Python?

# Why Python? [6]



- Minimalistic language
- Full range of object orientation
- Interactive mode (Python shell)
- Many (scientific) modules
- Non-commercial
- Big, active community (Stack Overflow, Real Python)

# Python Characteristics

# Python characteristics [6]



- Whitespace sensitive
- Object orientated (everything is an object)
- Typed and interpreted dynamically
- Functional
- Aspect orientated
- Interactive (ad-hoc scripts)
- Automatic memory management
- Easy readable
- “There should be one, and preferably only one, obvious way to do it.”

# Zen of Python I [6]



- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren't special enough to break the rules.
- Although practicality beats purity.
- Errors should never pass silently.

# Zen of Python II [6]

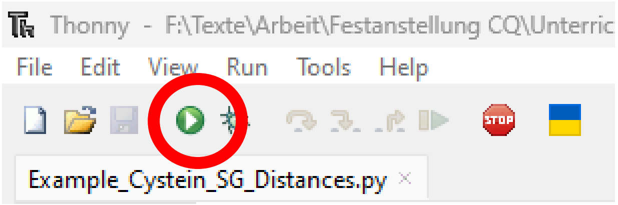


- Unless explicitly silenced.
- In the face of ambiguity, refuse the temptation to guess.
- There should be one-- and preferably only one --obvious way to do it.
- Although that way may not be obvious at first unless you're Dutch.
- Now is better than never.
- Although never is often better than right now.
- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.
- Namespaces are one honking great idea let's do more of those!

# Getting started

# Getting started I [7]

- Programming tutorials always start with the program called “Hello World”
- We will do the same
- You have to create a file hello.py with Thonny
- Open a new file and save it as hello.py
- Than add the following line to the script
  - `print(“Hello World”)`
- Save it again and start it by pressing the F5 Button on your keyboard or clicking on the green Arrow at the top of Thonny (red circle)



# Getting started II [7]



- Add the following commands to the file
  - `print("Jack and Jill went up a hill")`
  - `print("to fetch a pail of water;")`
  - `print("Jack fell down, and broke his crown,")`
  - `print("and Jill came tumbling after.")`
- Save the file and than run it again
- Now we look again at our file to see what happens when we execute it

# Getting started III – Program Run



```
print(“Hello World”)

print("Jack and Jill went up a hill")

print("to fetch a pail of water;")

print("Jack fell down, and broke his crown,")

print("and Jill came tumbling after.")
```

# Getting started IV – Comments



- You can add lines starting with a “#” to add comments that help you to understand what the program wants to do
- Always comment your code!!!
- Not only does it help you if you come back to work on a program later but it’s also important for sharing programs in a work group if you try to tackle a problem with more people than just you
- To generate comments that span multiple lines use:
  - `“””`
  - `Comment1`
  - `Comment2`
  - `“””`

# Exercise 02: Simple printing



- Write a program that prints out at least 5 of the following:
  - Your full name
  - Your birthday
  - Your study
  - A simple sentence
  - The shout “STOP!!!” (use “STOP!!!” inside the brackets of print)
  - The name of this course

# A standard python script

# Structure of Python scripts



- All Python scripts should follow the same structure as shown below

1. Imports of external modules / scripts
2. Declaration of global variables
3. Definitions of your own classes
4. Definitions of your own functions
5. The general algorithm that handles the task

- Over the course of the next weeks we will discuss all five points above in detail
- However we won‘t stick to the order as it‘s shown here and instead will go in order of complexity

# Definitions

# Definitions I



- A **Program** is something that can be executed by the computer to fullfill a certain task
- A **Script** is a text file that contains (parts of) the source code for a specific program
- The **Source Code** is the “text” written in our scripts
- **Compiler** create programs by converting scripts
- **Interpreter** read through scripts and execute them line by line
- We can write scripts with the help of **Programming Languages** that allow us to hand over commands to the computer

# Definitions II



- A **Graphical User Interface** allows the user to interact with the program by using the mouse
- **Variables** are placeholders for values in our scripts
- **Datastructures** are collections of variables that follow specific rules
- There are **Functions** that fullfill specific tasks that we have to use multiple times during the execution of our program
- Everything we write into brackets after a function name is called **Argument**
- Functions that work on specific data structures are called **Methods**
- To execute methods we need the **Point Notation**, that is used with the syntax datastructure.method()

# Definitions III



- We can define our own datastructures in form of **Classes**
- With the help of **Branchings** we can determine which parts of our script should be executed depending on certain conditions
- If we need to repeat a specific task over and over again, we can use a **Loop** to repeat parts of our source code
- **Built-in** functions and datastructures are part of the general python installation
- We can increase the number of available functions and datastructures with the help of **External Modules**
- To get access to external modules we need to **Import** them

# Definitions IV



- There are **Keywords** that fullfill specific tasks or signify the start of a specific programming construct like a loop or an if statement

# Literature




1. https://en.wikipedia.org/wiki/Computer_programming (Last edited: 21.03.2023, Last visited: 03.04.2023)
2. https://en.wikipedia.org/wiki/Ada_Lovelace (Last edited: 29.03.2023, Last visited: 03.04.2023)
3. https://en.wikipedia.org/wiki/A-0_System (Last edited: 21.02.2023, Last visited: 03.04.2023)
4. https://en.wikipedia.org/wiki/Grace_Hopper (Last edited: 01.04.2023, Last visited: 03.04.2023)
5. https://en.wikipedia.org/wiki/Fortran (Last edited: 30.03.2023, Last visited: 03.04.2023)
6. Presentation “Einführung in Python from the Friedrich-Schiller Universität Jena in the year 2019 (Download PDF: 25.1.2022)
7. https://en.wikibooks.org/wiki/Non-Programmer%27s_Tutorial_for_Python_3 (Last edited: 05.09.2022, Last visited: 02.01.2023)